

Expense Tracker

CIS 25 Final Project

Presented by: Elizabete Roschel



Project Overview

What is it?

The Expense Tracker is a robust command-line application designed to streamline personal financial management. It provides users with an intuitive interface to manage their transactions efficiently.

Core Functionality

Users can seamlessly add new transactions, view existing records, and apply filters to categorize their financial activities. This ensures a clear and organized overview of all income and expenses.

Data Persistence

A key feature of the application is its persistent data storage, achieved through efficient file I/O operations. This means all transaction data is securely saved and loaded, ensuring no loss of information between sessions.

Core Features



Add Transactions

Users can easily input new financial records, including the date, a detailed description, the relevant category, the monetary amount, and whether it's an income or an expense. This comprehensive input ensures accurate tracking.



Filter by Category

The application allows for precise filtering of transactions based on their assigned categories. This feature enables users to quickly analyze spending or income within specific areas, providing granular insights into their financial habits.



Summary Report

A vital component is the summary report, which provides a clear breakdown of total income, total expenses, and the overall balance. This report offers a high-level financial snapshot, aiding in quick assessment of financial health.

Data Storage

How Data is Stored

The Expense Tracker utilizes a simple yet effective method for data persistence: a plain text file named **data.txt**. This approach ensures portability and ease of access to the stored information.

Each transaction is meticulously recorded within this file, with specific fields separated by delimiters to maintain structural integrity. The data points stored for each transaction include:

- Date of the transaction
- A brief description
- The assigned category
- The monetary amount
- A boolean flag indicating if it's income (isIncome)

The application is designed to automatically save all current transaction data to **data.txt** upon exiting and load existing data from the file each time it is launched. This ensures that all financial records are consistently preserved across different sessions.



Classes and Structures



Transaction Class

This class serves as the fundamental building block for managing individual financial entries. It encapsulates all the essential details of a single transaction, ensuring data integrity and consistency.

- **Date:** Records the specific date when the transaction occurred.
- **Description:** Provides a brief textual explanation of the transaction.
- **Category:** Assigns a predefined category to the transaction for filtering and reporting.
- **Amount:** Stores the numerical value of the transaction.
- **isIncome:** A boolean flag indicating whether the transaction represents income (true) or an expense (false).



Tracker Class

The Tracker class acts as the central orchestrator of the application's functionality. It is responsible for managing a collection of **Transaction** objects and handling all interactions with the persistent data storage.

- **Transaction Vector:** Internally maintains a dynamic array (vector) of **Transaction** objects, allowing for efficient addition, deletion, and retrieval of transactions.
- **File I/O Management:** Handles the reading of transaction data from the **data.txt** file upon application startup and the writing of updated transaction data back to the file upon exit. This ensures that all changes are saved and available for future sessions.
- **Core Operations:** Implements methods for adding new transactions, filtering transactions by category, and generating summary reports, leveraging the data stored within the transaction vector.

Adding a Transaction (Code Example)

How it Works

The process of adding a new transaction is designed to be user-friendly and straightforward. When a user opts to add a transaction, the application prompts them to input several key details.

- **User Input:** The user is guided to enter the date, a brief description, the category, the amount, and to specify whether the transaction is an income or an expense.
- **Data Validation:** (Future Improvement) While not fully implemented in the current version, future iterations will include robust input validation to ensure data integrity and prevent errors.
- **Vector Integration:** Once all details are provided, a new **Transaction** object is created using the input data. This newly created object is then appended to the internal vector of **Transaction** objects managed by the **Tracker** class.
- **Persistence:** The updated vector, now containing the new transaction, will be saved to the **data.txt** file when the application is closed, ensuring the transaction is permanently recorded.

```
• cpp3 lines
• Click to expand
• void Tracker::addTransaction(const Transaction& t) {
•   transactions.push_back(t);
•   ...
• }
```

Summary Report (Code Example)

How it Works

The summary report feature provides users with a quick and comprehensive overview of their financial status. This functionality is crucial for understanding overall income, expenses, and the resulting balance.

- **Calculation Logic:** The application iterates through all stored **Transaction** objects within the internal vector. For each transaction, it checks the **isIncome** flag. If true, the amount is added to the total income; otherwise, it's added to the total expenses.
- **Balance Calculation:** The net balance is then computed by subtracting the total expenses from the total income.
- **Display:** Finally, the calculated total income, total expenses, and the net balance are clearly displayed to the user in a formatted output, providing an immediate financial snapshot.

```
void Tracker::summaryReport() {
    double income = 0, expenses = 0;
    for (const auto& transaction : transactions) {
        if (transaction.isIncome) {
            income += transaction.amount;
        } else {
            expenses += transaction.amount;
        }
    }

    std::cout << "\n--- Summary Report ---\n";
    std::cout << "Total Income: $" << income << std::fixed <<
    std::setprecision(2) << "\n";
    std::cout << "Total Expenses: $" << expenses << std::fixed <<
    std::setprecision(2) << "\n";
    std::cout << "Balance: $" << (income - expenses) << std::fixed <<
    std::setprecision(2) << "\n";
    std::cout << "-----\n";
}
```

Demo Screenshot:

PROBLEMS OUTPUT TERMINAL

Category: food
Amount: \$4.00
1. Income 2. Expense: 2

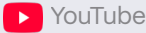
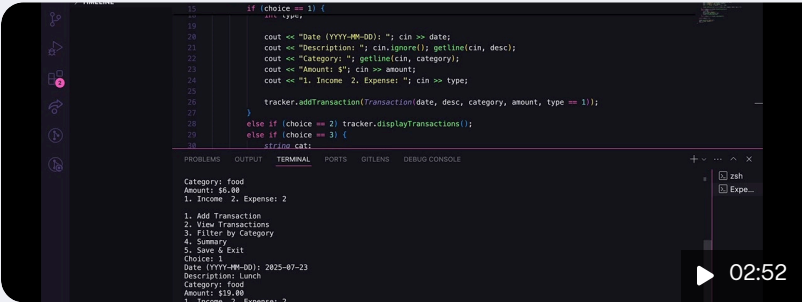
1. Add Transaction
2. View Transactions
3. Filter by Category
4. Summary
5. Save & Exit

Choice: 4
Total Income: \$0
Total Expenses: \$4
Balance: \$-4

1. Add Transaction
2. View Transactions
3. Filter by Category
4. Summary
5. Save & Exit

Choice: 5
Saved. Goodbye!

DEMO:



CIS-25 FINAL PROJECT DEMO



02:52

Future Improvements



Export to CSV

Implement functionality to export all transaction data into a CSV (Comma Separated Values) file. This will allow users to easily import their financial data into other spreadsheet software or accounting tools for further analysis or record-keeping.



Monthly/Category Analytics

Develop advanced analytical features to provide deeper insights into spending patterns. This includes generating reports that break down income and expenses by month and by specific categories, helping users identify trends and areas for financial adjustment.



GUI using Qt

Transition the command-line interface to a modern Graphical User Interface (GUI) using the Qt framework. A GUI will enhance user experience with visual elements, intuitive navigation, and a more interactive environment, making the application more accessible and user-friendly.



More Input Validation

Strengthen the input validation mechanisms to ensure that all user entries are accurate and in the correct format. This will prevent data corruption, improve the reliability of the application, and provide clear feedback to users on invalid inputs.

